

Interview Task: Caching Strategy & Consistency (.NET)

Space: Engineering Hiring | **Version:** 1.0 | **Last Updated:** April 2026

Tags: interview-task | senior-level | backend | .NET 8

Table of Contents

- 1. [General Information](#)
- 2. [Goal of the Assignment](#)
- 3. [Business Scenario](#)
- 4. [Part 1 – Caching Layer \(MANDATORY\)](#)
- 5. [Part 2 – Cache Invalidation \(MANDATORY\)](#)
- 6. [Part 3 – Expiration Strategy \(MANDATORY\)](#)
- 7. [Part 4 – Advanced Requirement \(MANDATORY\)](#)
- 8. [Explicitly Out of Scope](#)
- 9. [Submission](#)
- 10. [What We Evaluate](#)

General Information

This assignment evaluates your architectural and design skills. Business logic is intentionally simple.

Item	Details
Level	Senior .NET Developer
Focus	Dependency Injection, Service Lifetimes, Concurrency & State
Tech Stack	.NET 8, ASP.NET Core Web API
Allowed Tools	Documentation, Google, Stack Overflow, AI tools
Important	You must justify caching decisions

Goal of the Assignment

This assignment evaluates your ability to:

- Design a proper caching layer
- Avoid common cache bugs
- Handle cache invalidation
- Prevent stale data issues
- Understand memory vs distributed cache
- Apply correct lifetime & expiration strategy

Business logic is simple. Architecture & correctness are the focus.

Business Scenario (Minimal & Fixed)

You are implementing a **Product Catalog API**.

Endpoints:

```
GET  /api/products/{id}
POST /api/products
PUT  /api/products/{id}
```

Products are stored in an in-memory repository (`Dictionary`).

Goal: Improve read performance using caching.

Part 1 – Caching Layer (MANDATORY)

You must implement:

- A caching abstraction (e.g., `IProductCache`)
- Use `IMemoryCache` OR a custom abstraction
- Apply caching only for `GET`

Requirements:

- Cache key must be deterministic
 - Cache expiration strategy must be defined
 - Avoid caching `null` accidentally (your design decision)
-

Part 2 – Cache Invalidation (MANDATORY)

You must handle:

- Product update
- Product creation

Cache must:

- Invalidate or refresh correctly
- Avoid stale reads
- Not require application restart

Demonstrate correctness via a simple test scenario.

Part 3 – Expiration Strategy (MANDATORY)

You must implement at least one:

- Absolute expiration
- Sliding expiration

You must justify your choice during the defense interview.

Part 4 – Advanced Requirement (MANDATORY)

Add **one** of the following:

Option A:

- Cache stampede prevention (double-check locking or `GetOrCreateAsync` pattern)

Option B:

- Simulate distributed cache via interface abstraction

Option C:

- Implement manual cache invalidation endpoint

Choose one and be ready to justify your decision.

Explicitly Out of Scope

Do NOT implement:

- Redis (optional abstraction only)

- Database

- Third-party caching libraries

- Complex persistence

- Background refresh services

Focus on fundamentals.

Submission

Submit a link to a GitHub repository.

- The repository must be **public** (or shared with the interviewer)
- README must include:
 - How to run
 - Example request flow
 - Example demonstrating cache hit/miss
 - Brief notes on key design decisions
- No lengthy design explanation required — architectural decisions are covered in the oral defense

What We Evaluate

Your submission will be scored across six dimensions. Understanding these will help you make intentional design decisions.

1. **Spec-Driven Development (SDD):** How you broke down the requirements before coding, whether you used AI tools to assist (and how critically), and whether your decisions are traceable to the spec. If you used AI assistance, include a `PROMPTS.md`, `AI_USAGE.md`, or `DECISIONS.md` file documenting your process.
1. **Architecture:** How well your code is structured — separation of concerns between controller, service, and cache layers; use of interfaces and dependency injection; code organization and extensibility.
1. **Security:** Awareness of caching-specific security risks such as cache poisoning, accidental exposure of sensitive data in cache entries, improper key design that could allow cross-user data leakage, and input validation on all endpoints.
1. **Performance:** Correctness and efficiency of your caching strategy at scale — cache hit/miss ratio design, prevention of cache stampede under concurrent load, and the reasoning behind your expiration choices.

1. **Observability:** Whether your implementation makes cache behavior visible — structured logging of cache hits and misses, error visibility, and the ability to diagnose stale data or invalidation failures without restarting the application.
1. **Tests:** Your testing strategy — at minimum one test demonstrating cache hit/miss behavior, coverage of invalidation correctness, edge case handling (nulls, missing resources), and clear mocking of cache dependencies.

Generated from Assignment | Engineering Hiring | INTERNAL ONLY